Universidad Autónoma de Sinaloa

Licenciatura en Ingeniería de Software

Tarea 10: virtualización nativa, persistencia y seguridad en contenedores

Materia:

Administración de Sistemas

Nombre del Profesor:

Dr. Herman Geovany Ayala Zúñiga

Nombre del alumno:

Espiritu Manzanarez Diego Manuel

Fecha:

25/04/2026

➢ Introducción y Arquitectura o lo que trate cada práctica

- Historial de Cambios:

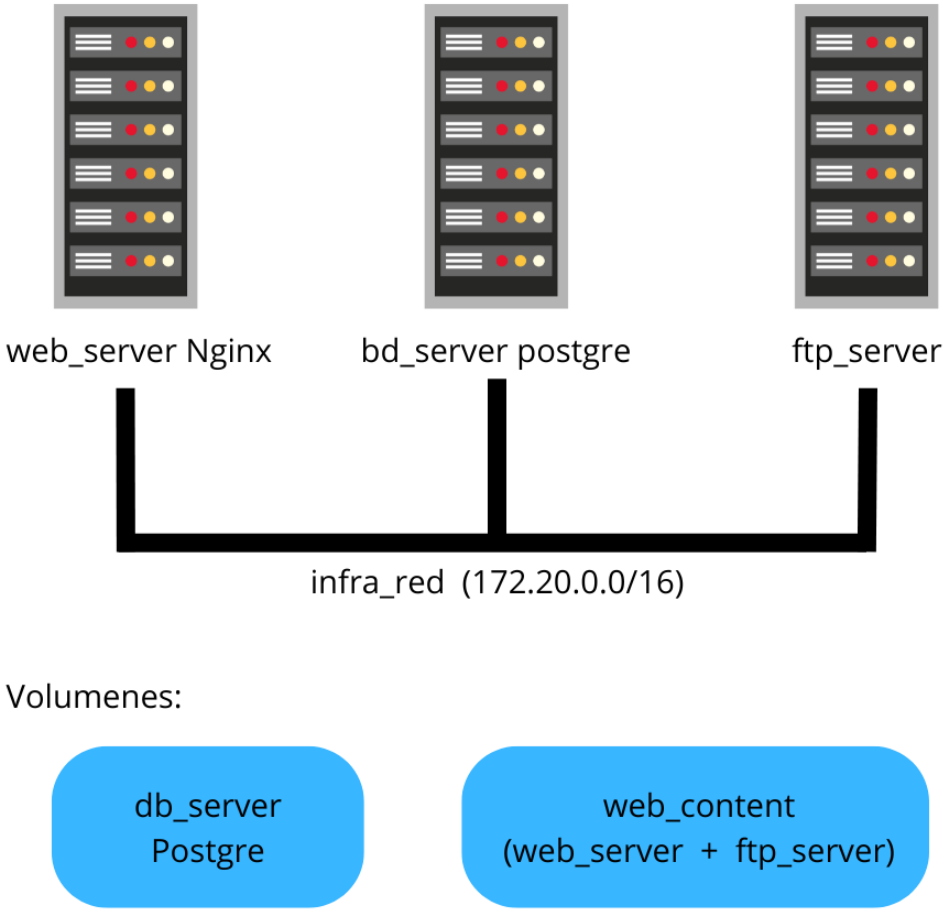
Repositorio: https://github.com/Diegoesp-0/Administracion-de-Sistemas

Versión	Fecha	Descripción del Cambio	Relación con Git
10.0	25/04/2026	Se crearon las carpetas para poner los archivos y documentación.	Tarea 10: Agregar carpetas
10.1	25/04/2026	Se crearon carpetas donde se guardarán los archivos para la página web.	Tarea 10: Carpetas para web
10.2	25/04/2026	Se guardaron los archivos index, css y el logo.	Tarea 10: agregar diseño del servidor web
10.3	25/04/2026	Se creo el archivo donde se instalará el Docker	Tarea 10: Modulo instalación
10.4	25/04/2026	Se agrego el modulo donde se crea la infra red.	Tarea 10: Modulo para la infra red
10.5	25/04/2026	Se creo el módulo para los volúmenes del docker.	Tarea 10: Modulo volumenes
10.6	25/04/2026	Este modulo es el encargado del docker web.	Tarea 10: Modulo para el web
10.7	25/04/2026	Se agrego un modulo encargado de la BD.	Tarea 10: Modulo para la base de datos
10.8	25/04/2026	Se creo el modulo que se encargara del FTP.	Tarea 10: Modulo FTP
10.9	25/04/2026	Se agrego el archivo que moverá todo.	Tarea 10: Primera version del main
10.10	25/04/2026	Se agrego un mensaje para poder tener una buena progresión.	Tarea 10: Arreglos en install
10.11	25/04/2026	Se agrego un .env el cual tendrá las credenciales.	Tarea 10: Se agrego .env

- Objetivo:

Migrar los servicios de prácticas anteriores (web, base de datos y transferencia de archivos) a contenedores Docker personalizados, garantizando persistencia de datos, aislamiento de red y límites de recursos mediante automatización con scripts Bash y Dockerfile.

- Diagrama de Topología:



➢ Guía de Uso de los Scripts

- Requisitos previos:

- Tener openSUSE con acceso a internet y permisos sudo.
- Tener los puertos 21, 8080 y 21000-21010 disponibles.
- Tener un adaptador puente enp0s9.

➢ Instrucciones de Ejecución:

1- Al tener el script pondremos el comando “bash practica10.sh -i”, esto instalara todo lo necesario para poder realizar la práctica.

```
Tarea-10/Scripts> bash practica10.sh -i
```

2- Una vez terminado el proceso, pedirá ejecutar el comando “newgrp docker” para luego ejecutar nuevamente “bash practica10.sh -i”

```
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-10/Scripts> bash practica10.sh -

==== Instalacion completa ====

[INFO] Verificando instalacion de Docker...
[INFO] Docker no encontrado, instalando...
[OK] Docker instalado correctamente
[INFO] Verificando cron...
[INFO] Instalando cronie...
[OK] Cronie instalado y activo
[INFO] Verificando servicio Docker...
[INFO] Iniciando servicio Docker...
[OK] Servicio Docker iniciado
[INFO] Verificando grupo docker...
[INFO] Usuario agregado al grupo docker
[AVISO] Ejecuta: newgrp docker y vuelve a correr el script
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-10/Scripts> newgrp docker
```

3- Una vez ejecutado por segunda vez el comando podremos ver que están activados los dockers y listos para usarse.

```
==== Estado de contenedores ====

NAMES        STATUS          PORTS
ftp_server   Up 2 seconds    0.0.0.0:21->21/tcp, [::]:21->21/tcp, 0.0.0.0:21000-21010->21000-21010/tcp, [::]:21000-21010->21000-21010/tcp
db_server    Up 10 seconds   5432/tcp
web_server   Up 31 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp


==== Uso de recursos ====

CONTAINER ID   NAME         CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O        PIDS
19f6ba304cff   ftp_server   0.07%     1.117MiB / 512MiB   0.22%     858B / 126B     12.3kB / 512kB   3
80ab0926d3a0   db_server    0.25%     39.85MiB / 512MiB   7.78%     1.59kB / 126B   0B / 73.7MB      6
24950f03a02a   web_server   0.00%     3.984MiB / 512MiB   0.78%     3.08kB / 126B   4.1kB / 303kB    4
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-10/Scripts>
```

4- Para verficiar que todo funcione bien tienes la opción de poner el comando “bash practica10.sh -v”

```
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-10/Scripts> bash practica10.sh -v

==== Estado de contenedores ====

NAMES        STATUS          PORTS
ftp_server   Up 15 minutes   0.0.0.0:21->21/tcp, [::]:21->21/tcp, 0.0.0.0:21000-21010->21000-21010/tcp, [::]:21000-21010->21000-21010/tcp
db_server    Up 16 minutes   5432/tcp
web_server   Up 16 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp


==== Uso de recursos ====

CONTAINER ID   NAME         CPU %     MEM USAGE / LIMIT   MEM %     NET I/O           BLOCK I/O        PIDS
19f6ba304cff   ftp_server   0.10%     1.27MiB / 512MiB    0.25%     13.9kB / 15.9kB   12.3kB / 520kB   3
80ab0926d3a0   db_server    0.03%     42.9MiB / 512MiB    8.38%     7.64kB / 126B     0B / 76.6MB      6
24950f03a02a   web_server   0.00%     4.078MiB / 512MiB   0.80%     25.6kB / 224kB    4.1kB / 303kB    4
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-10/Scripts> _
```

➢ Bitácora de Desarrollo y Configuración:

- Explicación de Script:

El script practica10.sh gestiona una infraestructura de tres contenedores Docker (servidor web nginx, base de datos PostgreSQL y servidor FTP) sobre openSUSE. Su lógica principal se basa en la idempotencia: antes de crear cualquier recurso, cada función verifica si este ya existe, lo que permite ejecutar el script múltiples veces sin errores ni duplicados.
Al iniciarse, el script calcula su propia ubicación para construir rutas relativas y carga primero el archivo de credenciales claves.txt, asegurando que las variables de usuario y contraseña estén disponibles para todos los módulos que se cargan después. La función instalar_docker verifica la presencia del binario de Docker mediante command -v, y si no existe lo instala con el gestor de paquetes de openSUSE. También comprueba que el demonio esté activo con systemctl is-active, que el usuario pertenezca al grupo docker consultando groups, que cronie esté instalado para poder programar tareas automáticas, y abre los puertos necesarios en el firewall en una sola instrucción.
La función crear_red crea una red bridge personalizada llamada infra_red, lo que permite que los contenedores se comuniquen entre sí por nombre en lugar de por IP. La función crear_volumenes crea dos volúmenes persistentes: db_data para que los datos de PostgreSQL sobrevivan reinicios, y web_content que es compartido entre el servidor web y el FTP, permitiendo que los archivos subidos por FTP aparezcan automáticamente en el sitio web.
Para el contenedor web, el script primero construye una imagen personalizada basada en nginx:alpine que ejecuta los procesos como un usuario sin privilegios y oculta la versión del servidor por seguridad. Al levantar el contenedor distingue tres estados: si ya está corriendo no hace nada, si está detenido lo inicia, y si no existe lo crea y copia los archivos web al volumen compartido. La base de datos PostgreSQL sigue la misma lógica, y al crearse por primera vez instala automáticamente un cron job que ejecuta pg_dump dentro del contenedor cada día a las 2:00 AM, guardando los respaldos en /opt/docker_backups/ y conservando solo los últimos 7 archivos. El servidor FTP utiliza ip addr show enp0s9 para obtener la IP real del adaptador puente de la VM y la pasa como variable de entorno al contenedor, resolviendo el problema del modo pasivo FTP que por defecto anunciaría la IP interna de Docker en lugar de la IP accesible desde el exterior.

- Evidencias de Configuración:

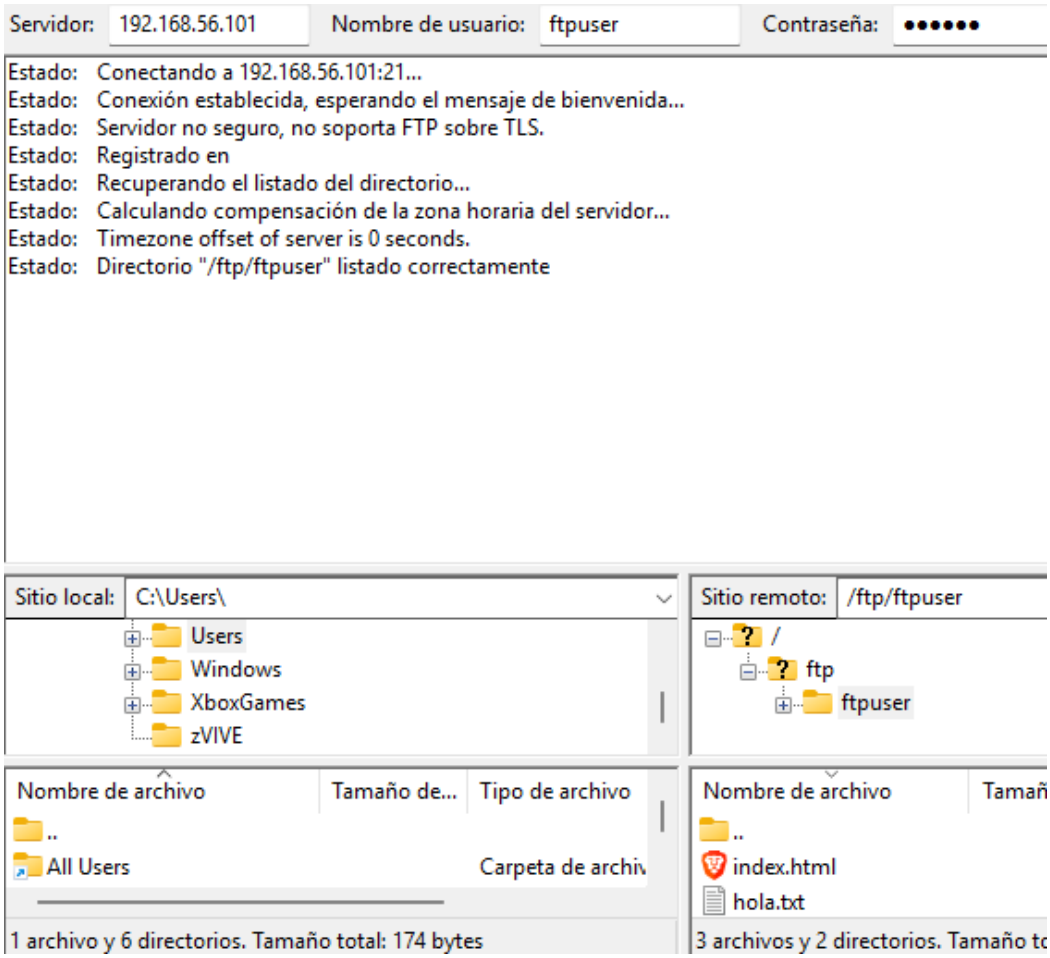
1- Infra red creada con los dockers vinculados.

```
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-10/Scripts> docker network inspect infra_red
[
    {
        "Name": "infra_red",
        "Id": "1a5b0c28a5b615624184ec1aeb88957901a11855aba162bebf70f1331ee0f373",
        "Created": "2026-04-27T23:46:50.248587662+02:00",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.20.0.0/16"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "19f6ba304cff64a4377ed8b8d50b52f40f7b5db799dad9167e83161ba8a9e9f2": {
                "Name": "ftp_server",
                "EndpointID": "aac48f13f6b547364fef6ebd37bb3aba84ee0d6351a4c9a547665338eb77fbb4",
                "MacAddress": "2e:bc:cd:d7:43:24",
                "IPv4Address": "172.20.0.4/16",
                "IPv6Address": ""
            },
            "24950f03a02af7c5358f83aada5db823d86156b7a0f2d54409d9a7f78245bcc7": {
                "Name": "web_server",
                "EndpointID": "3d0704af72d0f217162514298971d025df4a8fbf3042fb5e516b1c75f195d020",
                "MacAddress": "fe:a9:9f:8c:e5:fa",
                "IPv4Address": "172.20.0.2/16",
                "IPv6Address": ""
            },
            "d04291160f8c91d38430cf5610d0ffe4fe62baa8bf4e09e1a40f7f6748a09c24": {
                "Name": "db_server",
                "EndpointID": "b3f586ff88c2792f6e1f7fb165625cc1ed0da87283c89657911dddc3e7cb6d3c",
                "MacAddress": "e6:27:c2:43:c4:14",
                "IPv4Address": "172.20.0.3/16",
                "IPv6Address": ""
            }
        },
        "Options": {},
        "Labels": {}
    }
]
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-10/Scripts>
```

2- Comprobación de Dockers existentes.

```
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-10/Scripts> docker volume ls
DRIVER    VOLUME NAME
local     db_data
local     web_content
Espiritu@localhost:~/Administracion-de-Sistemas/Tarea-10/Scripts>
```

3- Conexión con FileZilla.



➢ Conclusiones y Referencias

- Lecciones Aprendidas:

Aprendí que son los dockers exactamente y que usos y ventajas tiene tener uno ya sea en tu base de datos o almacenamiento, tuve como complicación la red para el ftp pero usando practicas pasadas pude solucionarlo y poder terminar la práctica.

- Bibliografía:

https://docs.docker.com/reference/cli/docker/container/run/

https://nginx.org/en/docs/http/ngx_http_core_module.html#server_tokens